



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

ESTUDIANTE

Kevin Abiud Vera Zambrano

DOCENTE

Ingeniero José Naranjo

ASIGNATURA

Práctica en gestión de la información

PARALELO

Séptimo Semestre.

Grupo 2



CRUD en .NET Core con SQL Server: Implementación Práctica

Definición y propósito.

El acrónimo CRUD hace referencia a las cuatro operaciones fundamentales para la gestión de datos persistentes: Crear (Create), Leer (Read), Actualizar (Update) y Eliminar (Delete). En el contexto de .NET Core y SQL Server, la implementación práctica de un CRUD consiste en desarrollar los mecanismos necesarios en una aplicación para interactuar con la base de datos de manera eficiente, utilizando herramientas modernas como Entity Framework Core.

El propósito principal de esta arquitectura es proporcionar una capa de acceso a datos que abstraiga la complejidad de las consultas directas y permita interactuar con la base de datos relacional utilizando objetos de C# fuertemente tipados. Esto resuelve el problema clásico de desconexión entre la organización relacional de la base de datos y la organización basada en objetos del software.

Componentes principales.

- **Modelos:** Son clases simples de C# que representan la estructura de los datos. Cada propiedad de la clase suele corresponder a una columna en la tabla de la base de datos.
- **DbSet:** Es una colección genérica que representa un conjunto de entidades específicas en el contexto. Es la puerta de entrada para operaciones de consulta y guardado, equivalente conceptualmente a una tabla o vista de la base de datos.



- **DbContext:** Es el componente central más importante, ya que representa una sesión con la base de datos. Se encarga de gestionar conexiones, configurar mapeos, rastrear cambios y coordinar transacciones al salvar los datos.

Flujo de trabajo.

La implementación estándar para lograr un CRUD funcional sigue un ciclo de vida definido:

1. **Configuración Inicial:** Instalar dependencias requeridas y configurar la cadena de conexión en la aplicación apuntando a SQL Server.
2. **Definición de Entidades y Contexto:** Programar las clases del Modelo y registrar el DbContext.
3. **Migraciones:** En un enfoque Code First, crear las clases en C# primero y luego generar automáticamente la base de datos. Utilizar comandos para generar scripts SQL que actualicen el esquema.
4. **Operaciones de Datos:**
 - a. **Crear:** Añadir instancias al DbSet y guardar cambios.
 - b. **Leer:** Ejecutar consultas LINQ para recuperar datos.
 - c. **Actualizar:** Modificar propiedades del objeto rastreado y guardar cambios.
 - d. **Eliminar:** Remover el objeto del DbSet y procesar la transacción.

Ventajas.

- Aumento drástico de la productividad al reducir el código repetitivo.
- Seguridad por defecto; las consultas LINQ se parametrizan, neutralizando ataques de SQL Injection.



- Abstracción de alto nivel, evitando cadenas de texto SQL propensas a errores tipográficos.

Desventajas.

- Sobrecarga de rendimiento (overhead) debido al proceso de compilación de consultas y materialización de objetos.
- No es el enfoque ideal para consultas masivas o reportes sumamente complejos con múltiples subconsultas.
- Requiere una curva de aprendizaje inicial para entender a fondo Entity Framework y LINQ.

Casos de uso.

La creación de un CRUD bajo esta arquitectura es el pilar para operaciones de negocio cotidianas. Es sumamente recomendado para aplicaciones nuevas de negocio, iteraciones rápidas de startups y proyectos con esquemas de datos cambiantes. Ejemplos reales incluyen el desarrollo del backend (APIs REST) para sistemas de inventario, gestores de recursos humanos, o plataformas escolares, donde el mantenimiento de catálogos y entidades estructuradas es constante y la velocidad de desarrollo es prioridad.

Mejores prácticas.

- **Uso de DTOs (Data Transfer Objects):** Nunca expongas las entidades de base de datos directamente en las respuestas o peticiones de los endpoints. Utiliza DTOs para separar la capa de presentación de la capa de datos, previniendo la exposición de campos sensibles.



- **Paginación y Filtrado en Base de Datos:** Al implementar el método de "Lectura" para colecciones grandes, no se debe traer toda la tabla a la memoria de la aplicación. Es vital utilizar los métodos `.Skip()` y `.Take()` junto con LINQ para que la paginación se traduzca a SQL puro y el servidor de base de datos solo devuelva los registros estrictamente necesarios.
- **Manejo Global de Excepciones:** En lugar de llenar cada método CRUD con bloques try-catch, implementa un Middleware global para capturar excepciones (como violaciones de llaves foráneas o caídas del servidor SQL). Esto permite centralizar el registro de errores y asegurar que la API siempre devuelva un formato de respuesta estándar en caso de fallo.